TECHNICAL APPENDICES

Reference matrices

The large reference tables that support the Nook Complete Product & Engineering Handbook v1.0. They are published separately so the handbook stays readable; they are part of the same package and share its status vocabulary.

READ THIS FIRST

Every table in these appendices is reproduced from the approved requirements package v0.2. Rule text, identifiers, statuses and blocking relationships are copied as written. **No rule was answered, added, removed or reworded** in producing this document, and no status was advanced.

Where a status reads OPEN or PROFESSIONAL CONFIRMATION, the decision has not been taken. The API is specified to refuse the dependent operation with `422 RULE_NOT_CONFIGURED` rather than apply a default.

Contents

A Business Rules Register 2
B Data Dictionary and Domain Model 12
C API Catalogue and Contracts 20
D Acceptance Criteria and Traceability 30
E Owner Decision Worksheet 37

APPENDIX A

Business Rules Register

Every business rule that must be decided by the hostel owner before the dependent capability can be built. Nothing here has been answered on the owner's behalf.

PROVENANCE

Reproduced from `02_BUSINESS_RULES_REGISTER.md` in the approved requirements package v0.2. Rule text, statuses and blocking relationships are copied as written. No rule was added, removed, reworded or resolved in producing this appendix.

Context confirmed by brief

ID	STATEMENT	STATUS	VALIDATION ACTION
BR-001	Initial operator is a partnership-operated ladies' hostel in Pune.	Confirmed by brief	Confirm legal entity and authorised decision-makers.
BR-002	Approximately 100 beds and 91–100 residents.	Confirmed by brief	Reconcile physical inventory and active resident register.
BR-003	Shared and private room arrangements exist.	Confirmed by brief	Collect approved room/bed labels and occupancy rules.
BR-004	Residents include students and working women.	Confirmed by brief	Do not infer different policies by category unless owner confirms them.
BR-005	Current operations use manual paper registers.	Confirmed by brief	Identify the authoritative register(s) for each data set.
BR-006	Payments may be cash, UPI and bank transfer.	Confirmed by brief	Document current evidence and reconciliation practice for each method.
BR-007	Security deposits are used.	Confirmed by brief	Confirm each resident's opening deposit balance.
BR-008	Admission requires manual approval.	Confirmed by brief	Identify authorised approver and required fields.
BR-009	Residents participate in room/bed selection.	Confirmed by brief	Confirm whether this is preference, selection, request or binding allocation.
BR-010	Notice periods and checkout procedures exist.	Confirmed by brief	Obtain actual policy/agreement and examples.
BR-011	The hostel has one owner and 1–2 staff members.	Confirmed by brief	Map real people to the permission matrix.
BR-012	Rent collection and outstanding tracking are the immediate problem.	Confirmed by brief	Establish current baseline and measurable success criteria with owner.

"Confirmed by brief" means the project brief stated it. It does not mean it has been checked against the hostel's records. Every validation action remains outstanding.

Rent terms, billing periods and due dates — `BR-REN-*`

ID	RULE/QUESTION	STATUS	EVIDENCE/DECISION REQUIRED	BLOCKS
BR-REN-001	What determines each resident's due date — fixed calendar date, admission-related date, agreement-specific date, or another rule?	OPEN	Owner explains current practice with examples across several residents. Do not assume admission-anniversary billing.	Charge generation, reminders, overdue report
BR-REN-002	What dates does each charge cover? Are start/end dates inclusive? How are recurring periods represented?	OPEN	Confirm cycle, boundaries and representation.	Billing and statements
BR-REN-003	At admission, when is the first payment due and what does it cover?	OPEN	Owner states the rule and gives an example.	First bill
BR-REN-004	How is admission partway through a billing period charged — full period, prorated, fixed agreed amount, or other?	OPEN	At least two worked examples with approved expected results.	First bill
BR-REN-005	What is the rent stop date at checkout? Is any amount due after physical departure because of notice terms?	OPEN	Confirm rent-stop rule and any proration. Do not infer.	Final bill, final settlement
BR-REN-006	If a due date is the 29th, 30th or 31st, what happens in a shorter month? Does any resident use a different convention?	OPEN	Confirm date arithmetic. Do not choose a convention by default.	Scheduler and billing calculation
BR-REN-007	Who may approve a rent change, from what effective date, and what happens to already-issued charges?	OPEN	Confirm notice, effective date, authorised actor. Old charges must not be silently rewritten.	Rental-term versions
BR-REN-008	Are grace periods and late fees used?	OPEN	If yes: amount or formula, start point, cap, waiver authority and applicable legal review. Leave disabled unless completely approved.	Penalties, overdue status, reminders
BR-REN-009	If a due date falls on a weekend or public holiday, what applies?	OPEN	Owner policy and any contractual constraint.	Due date, reminders, overdue status
BR-REN-010	Are food, electricity, laundry, damage, amenities or other fees billed?	OPEN	Identify which services exist, how measured, priced, approved and billed. Mark each as absent, manual, or in scope.	Additional charge lines
BR-REN-011	Are discounts, concessions, credits or write-offs provided? Who may apply one, on what evidence, and is second-person approval required?	OPEN	Types, effective dates, approval path, and whether applied to charges or settled as credit.	Adjustment model, adjustment API and audit
BR-REN-012	Are taxes added to resident charges?	PROFESSIONAL CONFIRMATION	Accountant/legal review of the actual accommodation supply and current rules.	Tax invoices and tax calculations
BR-REN-013	What documents does the hostel actually issue — invoices, statements, receipts, or another form?	OPEN	Confirm business practice, terminology and any applicable invoicing requirement, with the accountant where relevant.	Print/export layouts

Admission, occupancy and checkout — `BR-LIF-*`

ID	RULE/QUESTION	STATUS	EVIDENCE/DECISION REQUIRED	BLOCKS
BR-LIF-001	Who may approve an applicant, and at what point does the stay become active?	OPEN	Identify authorised approver, required fields and the activation event.	State model, move-in
BR-LIF-002	Is the resident's bed choice a preference, a request, a temporary hold or a binding allocation? Who has final authority?	OPEN	Owner explains current practice and the commitment point.	Booking/assignment semantics
BR-LIF-003	Is a temporary bed hold used? If so, what duration, what expiry behaviour, and is money collected to create it?	OPEN	Confirm whether holds exist at all before designing a booking feature.	Booking/hold feature
BR-LIF-004	At which exact event or date does a bed become unavailable, and become available again?	OPEN	Define status transitions and effective dates. A bed-release date must never be inferred from a checkout event.	Occupancy constraint, operational checkout
BR-LIF-005	What evidence, fields and approvals does move-in require, and what is its effective date?	OPEN	Required fields/documents, effective date, deposit status at move-in.	Move-in workflow, migration
BR-LIF-006	May a resident hold more than one bed during a transfer?	OPEN	Define transition/overlap policy.	Concurrency constraint
BR-LIF-007	Who approves a room or bed change, when does it take effect, and do rent or other charges change?	OPEN	Owner examples, approval path and charge treatment.	Stay/assignment changes, rental-term versions
BR-LIF-008	What are the mandatory resident fields?	OPEN	Necessity, purpose and alternatives for each field; data-minimisation review.	Forms and migration
BR-LIF-009	Which identity documents are required, if any?	**Unresolved + professional review**	Current policy, lawful verification process, accepted alternatives, access and retention.	Document module, BR-SEC-004
BR-LIF-010	What notice period applies, through which channel is notice given, when does it begin and end, and who acknowledges it?	OPEN	Written rules and examples. Identify whether resident or contract categories differ.	Final charges, checkout
BR-LIF-011	What constitutes formal operational checkout — key handover, inspection, meter confirmation, bed release?	OPEN	Confirm the operational steps. This is separate from financial settlement and is confirmed independently of it.	Checkout workflow
BR-LIF-012	How are overstay and early departure treated, and who approves exceptions?	OPEN	Rules, if any, and approval path. Leave any calculation disabled until approved.	Final settlement
BR-LIF-013	How are temporary absences and returns handled, and what is recorded?	OPEN	Owner examples and recordkeeping requirements, including whether charges pause.	Stay lifecycle, rent schedule

Payment entry, verification and receipt — `BR-PAY-*`

ID	RULE/QUESTION	STATUS	EVIDENCE/DECISION REQUIRED	BLOCKS
BR-PAY-001	Confirm the accepted payment methods: cash, UPI, bank transfer and any others.	OPEN	Owner confirms the actual list.	Payment capture
BR-PAY-002	What proves receipt for each method? What reference or document is required, and what is optional?	OPEN	Document current evidence practice per method.	Receipt status, reconciliation

ID	RULE/QUESTION	STATUS	EVIDENCE/DECISION REQUIRED	BLOCKS
BR-PAY-003	Can staff save a payment as reported/pending before independent confirmation? Who may mark it confirmed, and on what evidence?	OPEN	Confirm whether a pending state is needed, who confirms, and which reports include pending entries. A screenshot alone should not be assumed proof of settlement.	Status model, collection reports
BR-PAY-004	When is a receipt issued — at entry, after verification, or after allocation? What wording distinguishes reported from confirmed?	OPEN	Owner confirms timing and wording.	Receipt generation
BR-PAY-005	What constitutes a duplicate? Which fields are unique per method, and how is a suspected duplicate reviewed without rejecting legitimate similar transactions?	**Proposed rule; details needed**	Define unique reference fields by method and the review process. Keep the request idempotency key independent from a bank UTR.	Duplicate warnings
BR-PAY-006	How is an unidentified transfer handled?	OPEN	Who investigates, how it is held, and what resolves or returns it.	Reconciliation workflow
BR-PAY-007	Who counts and compares cash, how often, against which records, and who signs off a discrepancy?	OPEN	End-of-day or other cash-close process.	Cash reconciliation report
BR-PAY-008	Which bank or UPI source is treated as evidence, and who confirms receipt?	OPEN	Owner names the authoritative source.	Transfer reconciliation
BR-PAY-009	Who may correct a mistaken entry, reverse or reassign an allocation, or issue a refund? What reason, evidence and second approval are required?	**Proposed invariant: no silent overwrite or delete.** Authority unresolved	Owner approves the correction/reversal workflow and authority.	Correction UI/API, refund model
BR-PAY-010	Is cash balance or cashbook tracking needed in P0?	OPEN	Confirm whether daily cash-close is a core workflow.	Conditional expense/cash module

Payment allocation and opening balances — `BR-ALLOC-`, *`BR-OPEN-`*

ID	RULE/QUESTION	STATUS	EVIDENCE/DECISION REQUIRED	BLOCKS
BR-ALLOC-001	Can a payment be allocated to one charge or several? Can the resident specify the target bill?	OPEN	Confirm allocation policy and whether resident instructions govern.	Allocations
BR-ALLOC-002	If the payment does not name a charge, which bill is settled first?	**Unresolved. No oldest-first assumption.**	Owner defines priority, or authorises none. If no default is authorised, the system requires explicit staff selection.	Automatic allocation
BR-ALLOC-003	Is a surplus held as unallocated credit, returned, allocated by choice, or handled another way? Who approves?	OPEN	Confirm workflow, reporting and subsequent allocation rules.	Advance/overpayment handling
BR-ALLOC-004	May payments be recorded before they can be matched? What is shown as outstanding until matching or confirmation?	OPEN	Confirm whether unallocated receipts are permitted and how they report.	Unallocated receipt handling
BR-ALLOC-005	How is an allocation corrected after an error? Which entries remain visible, and who approves?	OPEN	Confirm correction path and approval.	Allocation reversal
BR-OPEN-001	Which registers or statements establish current residents, bed inventory, dues, payments, deposits and credits?	OPEN	Identify the authoritative source for each data set, and which wins when two disagree.	Migration mapping

ID	RULE/QUESTION	STATUS	EVIDENCE/DECISION REQUIRED	BLOCKS
BR-OPEN-002	At what cut-off date are dues, credits, confirmed payments and unconfirmed receipts measured?	OPEN	Define the cut-off and prevent double-counting a receipt already reflected in a balance.	Opening balances, go-live
BR-OPEN-003	How is each resident's opening position reconciled?	OPEN	Approve resident count, current stay/bed assignment, charges/dues, payments/allocations and deposit balance separately.	Reconciliation report
BR-OPEN-004	How is every missing, disputed, conflicting or estimated amount handled?	OPEN	Decide per item whether it must be resolved before cutover or may remain explicitly flagged outside the authoritative balance.	Discrepancy register
BR-OPEN-005	Who signs the reconciliation report before real-data cutover?	OPEN	Owner signs a report showing row counts, totals by category and all approved differences.	Go-live

Security deposits, deductions and refunds — `BR-DEP-*`

ID	RULE/QUESTION	STATUS	EVIDENCE/DECISION REQUIRED	BLOCKS
BR-DEP-001	How is the deposit amount determined — fixed, room-dependent, a multiple of rent, instalments, or agreement-specific?	**Unresolved. No default assumed.**	Owner states the rule.	Deposit terms
BR-DEP-002	When is the deposit due? Which methods, partial collection and receipt requirements apply?	OPEN	Current practice and allowed partial collection.	Deposit receipts
BR-DEP-003	How is deposit money recorded and accounted for separately from rent, and what statement should the resident and owner see?	**Unresolved + accountant confirmation where needed**	Confirm classification and presentation.	Deposit reporting
BR-DEP-004	Is the deposit refundable, and under what contract or policy conditions?	OPEN	Agreement/policy and exceptions.	Resident-facing text, settlement
BR-DEP-005	Which deductions are permitted? What evidence is needed, who estimates and approves, and how does a resident dispute one?	**Unresolved + professional review as needed**	Enumerate each permitted deduction type. No deductions until approved.	Deposit settlement
BR-DEP-006	May the deposit be applied against unpaid rent or other dues?	OPEN	If yes, confirm the agreement, legal and accounting basis and the approval path. **Do not automatically net off.**	Settlement calculation
BR-DEP-007	How should partial receipts, deductions, refunds and corrections affect the available deposit balance?	OPEN	Define the balance contract.	Deposit balance display
BR-DEP-008	What refund deadline and process apply? Who confirms the amount, who executes the transfer or cash refund, what evidence proves it, and what happens if disputed?	OPEN	Agreement/owner policy and legal review where relevant.	Refund due-date and tracking
BR-DEP-009	Who are the maker, reviewer and approver for a deduction or refund? Are there amount thresholds? May one person both initiate and approve?	OPEN	Single vs dual approval and thresholds.	Permission matrix, settlement
BR-DEP-010	How is damage documented?	OPEN	Inspection record, evidence, resident acknowledgement and dispute process.	Optional inspection workflow

Roles, access, documents, reports and operations — `BR-SEC-`, *`BR-REP-`*, `BR-OPS-*`

Role names below are the canonical names defined in `01_PRD.md` §4 and detailed in the permission baseline in `10_ARCHITECTURE_APPROVAL_REGISTER.md` §2. *(C-15)*

ID	RULE/QUESTION	STATUS	EVIDENCE/DECISION REQUIRED	BLOCKS
BR-SEC-001	Which real staff member holds which role, and what are the exceptions?	OPEN	Named list mapping each person to a proposed role.	Account provisioning, go-live
BR-SEC-002	Which roles may see balances, statements, payment references, collections and exports?	OPEN	Approve the role-action matrix. Owner/Partner, Property Manager, Finance Clerk/Cashier and Warden/Front Desk are answered separately.	Authorisation
BR-SEC-003	Who can create or adjust charges, confirm a payment, correct an allocation, issue a refund, change roles, export data or commit a migration?	OPEN	Owner approves each sensitive action independently.	Sensitive-action permissions
BR-SEC-004	Which documents are truly required for P0, why, on what legal basis, with what alternatives, permitted viewers and retention period?	**Unresolved. Default proposal is no binary upload in P0.**	Owner must prove necessity before upload is enabled. See ADR-012.	Document module
BR-SEC-005	Is Aadhaar required at all? Which lawful verification process and alternative IDs are accepted?	**Unresolved + professional confirmation**	Obtain professional and local confirmation before building verification functionality.	Identity verification
BR-SEC-006	Which verified contact channel is available for staff account recovery, or should recovery be owner-assisted? Who confirms a recovery request?	OPEN	Confirm the channel and the confirming person. See ADR-005.	Account recovery
BR-REP-001	What is the exact cutoff, included charge/payment statuses and meaning of "outstanding"?	OPEN	Define precisely.	Outstanding report
BR-REP-002	Is the collection report based on recorded date, receipt date, confirmation date or another event?	OPEN	Owner chooses the basis date.	Collection report
BR-REP-003	What does the deposit report include, and how is separation from rent collection confirmed?	OPEN	Define inclusion and separation.	Deposit report
BR-REP-004	What do active, reserved, unavailable and vacant mean for this hostel's actual process?	OPEN	Define each status against real practice.	Occupancy report
BR-REP-005	Which reports are essential daily and monthly?	OPEN	Rank reports and define inclusion/cutoff rules.	Dashboard/report acceptance
BR-OPS-001	Are reminders sent automatically?	OPEN	Channels, timing, template, opt-out/consent and approval.	Messaging automation
BR-OPS-002	Are expenses required for P0?	OPEN	Determine whether essential to the target operating workflow.	P0 scope

ID	RULE/QUESTION	STATUS	EVIDENCE/DECISION REQUIRED	BLOCKS
BR-OPS-003	How long are records and documents retained?	PROFESSIONAL CONFIRMATION	Legal/contractual/accounting review; define retention by category.	Retention jobs
BR-OPS-004	How will a resident request correction, deletion or access?	PROFESSIONAL CONFIRMATION	Map current and future statutory obligations to the service arrangement.	Privacy operations
BR-OPS-005	What data loss and downtime can the operator tolerate?	OPEN	Owner states tolerance, then translate to RPO/RTO.	Backup design, ADR-016
BR-OPS-006	Who investigates discrepancies and exceptions, and within what target response time?	OPEN	Name the person. Do not encode an SLA until agreed.	Escalation workflow
BR-OPS-007	What measurable criteria would let the owner stop duplicating the paper register?	OPEN	Agree measurable pilot success criteria.	RG-5 owner acceptance
BR-OPS-008	Who is authorised to accept opening records, security permissions, known limitations and cutover?	OPEN	Name the authorised approver.	Go-live sign-off

Proposed architecture invariants (not commercial rules)

The following are recommended for approval because they protect correctness independently of the hostel's chosen policy. They are **architecture invariants**: always required, never dependent on an owner decision, and tested as such. They are distinct from owner-policy expectations, whose expected results must be signed off before any test asserts them. `08_TESTING_AND_MIGRATION_PLAN.md` §5 applies the same distinction to the financial test suite. *(C-12)*

1. Money is stored as integer paise in a `BIGINT` column and carried across the API as a base-10 integer string with an explicit currency, never as binary floating point. See ADR-008. *(C-01)*
2. A payment allocation cannot exceed the valid amount available for allocation.
3. A payment and its allocations commit atomically.
4. Duplicate billing retries cannot create duplicate charges for the same intended billing event.
5. Financial corrections preserve an auditable trail; posted records are not physically updated or deleted.
6. Security-deposit balances are reported separately from rent receipts and revenue.
7. Concurrent operations cannot create conflicting active bed allocations under the owner-approved state definition.
8. Organisation-owned records cannot be accessed across organisation boundaries, including through lists, searches, reports, exports, imports, background jobs and file access.
9. A charge cannot be generated, and a policy-dependent financial operation cannot execute, while required business policy inputs remain unresolved. *(C-09)*
10. Destructive imports, bulk changes and financial corrections require preview, permission checks and an audit record.
11. An operation blocked by an unresolved policy returns an explicit, documented unconfigured-policy response and writes no partial record. It does not fall back to a default, and it does not fail silently. *(C-09, C-12)*

Identifier mapping from version 0.1

In version 0.1 this register and document 11 used overlapping `BR-` *identifiers for different rules. The table below maps every version 0.1 identifier to its canonical version 0.2 identifier. Both source columns are given because the same string meant different things in the two documents.* (C-17)*

V0.1 IN DOC 02	V0.1 IN DOC 11	CANONICAL V0.2	RULE
BR-REN-001	BR-REN-001	BR-REN-001	Due-date rule
BR-REN-002	BR-REN-002	BR-REN-002	Charge period
—	BR-REN-003	BR-REN-003	First due date at admission
BR-REN-004	BR-REN-004	BR-REN-004	Mid-period admission charge
BR-REN-005	BR-REN-005	BR-REN-005	Checkout rent stop
BR-REN-006	BR-REN-006	BR-REN-006	Calendar edge cases
BR-REN-007	BR-REN-007	BR-REN-007	Term changes
BR-REN-008, BR-REN-009	BR-REN-008	BR-REN-008	Grace periods and late fees
BR-REN-003	—	BR-REN-009	Due date on weekend/holiday
BR-REN-011	BR-REN-009	BR-REN-010	Other charges
BR-REN-010, BR-REN-013	BR-REN-010	BR-REN-011	Discounts, credits, write-offs and approval
BR-REN-012	—	BR-REN-012	Tax treatment
BR-REN-014	BR-REN-011	BR-REN-013	Invoice/statement/receipt format
BR-LIF-001	BR-LIF-001	BR-LIF-001	Admission approval and activation
BR-LIF-002	BR-LIF-002	BR-LIF-002	Bed choice semantics
BR-LIF-003	—	BR-LIF-003	Temporary bed hold
BR-LIF-004	BR-LIF-003	BR-LIF-004	Bed availability transitions
—	BR-LIF-004	BR-LIF-005	Move-in evidence
BR-LIF-005	—	BR-LIF-006	Overlapping allocations during transfer
BR-LIF-010 (part)	BR-LIF-005	BR-LIF-007	Room/bed change approval and charge effect
BR-LIF-006	—	BR-LIF-008	Mandatory resident fields
BR-LIF-007	—	BR-LIF-009	Required identity documents
BR-LIF-009	BR-LIF-006	BR-LIF-010	Notice requirement
BR-LIF-008	BR-LIF-007	BR-LIF-011	Formal operational checkout
—	BR-LIF-008	BR-LIF-012	Overstay and early departure
BR-LIF-010 (part)	—	BR-LIF-013	Temporary absence and return
—	BR-PAY-001	BR-PAY-001	Accepted methods
—	BR-PAY-002	BR-PAY-002	Evidence per method
BR-PAY-001, BR-PAY-002, BR-PAY-003	BR-PAY-003	BR-PAY-003	Payment status model and confirmation

V0.1 IN DOC 02	V0.1 IN DOC 11	CANONICAL V0.2	RULE
—	BR-PAY-004	BR-PAY-004	Receipt timing and wording
BR-PAY-008	BR-PAY-005	BR-PAY-005	Duplicate real-world reference
BR-PAY-007	BR-PAY-006	BR-PAY-006	Unidentified transfer
—	BR-PAY-007	BR-PAY-007	Cash reconciliation
BR-PAY-011	BR-PAY-008	BR-PAY-008	Transfer reconciliation
BR-PAY-009, BR-PAY-010	BR-PAY-009	BR-PAY-009	Corrections, reversals and refunds
BR-PAY-012	—	BR-PAY-010	Cashbook in P0
BR-PAY-004	BR-ALLOC-001	BR-ALLOC-001	Allocation choices
BR-PAY-005	BR-ALLOC-002	BR-ALLOC-002	Default allocation ordering
BR-PAY-006	BR-ALLOC-003	BR-ALLOC-003	Advance and overpayment
—	BR-ALLOC-004	BR-ALLOC-004	Unallocated receipts
—	BR-ALLOC-005	BR-ALLOC-005	Allocation reversal
—	BR-OPEN-001 … BR-OPEN-005	BR-OPEN-001 … BR-OPEN-005	Opening balances, unchanged
BR-DEP-001	BR-DEP-001	BR-DEP-001	Deposit amount
BR-DEP-002	BR-DEP-002	BR-DEP-002	Collection timing
—	BR-DEP-003	BR-DEP-003	Deposit classification
BR-DEP-003	BR-DEP-004	BR-DEP-004	Refundability
BR-DEP-004	BR-DEP-005	BR-DEP-005	Permitted deductions
BR-DEP-005	BR-DEP-006	BR-DEP-006	Offset against rent
—	BR-DEP-007	BR-DEP-007	Available balance contract
BR-DEP-006	BR-DEP-008	BR-DEP-008	Refund deadline and process
BR-DEP-007	BR-DEP-009	BR-DEP-009	Approval roles and thresholds
BR-DEP-008	—	BR-DEP-010	Damage documentation
—	BR-SEC-001	BR-SEC-001	Named staff to roles
BR-OPS-001	BR-SEC-002	BR-SEC-002	Financial visibility
—	BR-SEC-003	BR-SEC-003	Sensitive actions
BR-OPS-002	BR-SEC-004	BR-SEC-004	Documents required for P0
—	BR-SEC-005	BR-SEC-005	Identity verification
—	BR-SEC-006	BR-SEC-006	Account recovery channel
—	BR-REP-001 … BR-REP-004	BR-REP-001 … BR-REP-004	Report definitions, unchanged
BR-OPS-004	—	BR-REP-005	Essential daily/monthly reports
BR-OPS-003	—	BR-OPS-001	Automatic reminders
BR-OPS-005	—	BR-OPS-002	Expenses in P0

V0.1 IN DOC 02	V0.1 IN DOC 11	CANONICAL V0.2	RULE
BR-OPS-006	—	BR-OPS-003	Record retention
BR-OPS-007	—	BR-OPS-004	Resident data requests
BR-OPS-008	—	BR-OPS-005	Recovery tolerance
—	BR-OPS-001	BR-OPS-006	Discrepancy escalation
—	BR-OPS-002	BR-OPS-007	Pilot success criteria
—	BR-OPS-003	BR-OPS-008	Go-live sign-off authority

No rule present in either version 0.1 document was dropped in this consolidation.

APPENDIX B

Data Dictionary and Domain Model

Proposed entities, attributes, relationships and the integrity constraints the database is expected to enforce. Proposed throughout — no schema has been written.

> **PROVENANCE**
>
> Reproduced from `03_DOMAIN_AND_DATA_MODEL.md` (requirements package v0.2). `prisma/schema.prisma` in the repository declares no models, so nothing in this appendix is implemented.

High-level relationship map

```text
Organisation ──< Membership >── User
  │
  └──< Property ──< Room ──< Bed
  │                  │
  │                  └──< ResidentStay >── Resident
  │
  ├──< RentalTermsVersion
  ├──< Charge ──< ChargeLine
  │     ▲
  │     └──────< PaymentAllocation >────── Payment
  │
  ├──< DepositAccount ──< DepositEntry
  ├──< AssignmentHistory
  └──< LifecycleEvent

DocumentMetadata ──> organisation + permitted owner object (resident/stay)
AuditEvent ──> organisation + actor + target reference (with minimised payload)
ImportBatch ──< ImportRowResult
```

This is a conceptual map, not a strict schema diagram. Exact ownership and cardinalities — particularly whether deposit is per resident or per stay, and whether one stay can contain multiple concurrent bed assignments during a transfer — require approval. See BR-DEP-003 and BR-LIF-006.

Proposed entities

1 Organisation

Represents one independent hostel operator/customer account.

Purpose: Top-level tenant boundary for customers, memberships, properties, configuration and audit ownership.

Conceptual attributes: stable ID, display/legal name as necessary, status, created/updated timestamps, settings version.

Constraints: Every customer-owned resource must be associated with one organisation. The active organisation is derived from the authenticated server-side session and the user's verified membership. A client-supplied `organization_id` in a path, query, body or header is never an authorisation mechanism, and ownership is verified server-side on every access. Where a user belongs to more than one organisation, changing the active context requires an explicit authenticated switch operation that re-confirms membership. See ADR-007 and `04_API_SPECIFICATION.md` §2. *(C-05)*

2 User and Membership

`User` is an authenticated principal; `Membership` connects a user to an organisation and assigned role/permissions.

Why separate: One staff member may eventually work with multiple organisations, but permissions must be explicit per organisation. A user identity is not itself a grant of access to every organisation's data.

Conceptual attributes: user ID, login identifier, authentication status; membership ID, organisation ID, role/permission grants, status, invited/activated/revoked timestamps.

Session strategy: Decided as proposed in ADR-004 — server-side revocable sessions with opaque `HttpOnly` cookies. Session expiry values, MFA policy, recovery channel and the invitation flow remain **unresolved** (BR-SEC-006, ADR-005). *(C-02)*

Open decision: Whether custom permissions are needed beyond a small fixed set of roles (BR-SEC-002, BR-SEC-003).

3 Property

Represents a physical hostel/PG premises managed by an organisation.

Conceptual attributes: ID, organisation ID, name, address fields needed for operations, status, timezone, created/updated timestamps.

Initial scope: One property. The ownership relationship must already support multiple properties per organisation. This is a platform requirement, not optional headroom.

4 Room and Bed

`Room` belongs to a property. `Bed` belongs to a room, even if some room types are private/single occupancy; the design may represent one reservable unit as a bed/place so availability logic stays consistent.

Conceptual attributes: labels/codes, room type/sharing attributes, capacity metadata if needed, status.

Open decision: Whether the physical inventory has beds, rooms rented as a whole, temporary/extra beds or non-bed spaces. Confirm actual property layout before choosing the canonical allocatable unit (BR-003, BR-LIF-002).

5 Resident

Represents a person. Keep the resident profile separate from an individual stay to preserve history across repeat admissions.

Conceptual attributes: internal ID, minimal approved profile fields, lifecycle status, created/updated timestamps. Do not add fields such as an Aadhaar number merely because an ID might be collected operationally (BR-SEC-005).

Open decision: Minimum required personal data (BR-LIF-008), duplicate matching rules, and whether the hostel wants one resident record for repeat stays.

6 ResidentStay

Represents the resident's specific stay at a property, including admission, allocation, current/historical lifecycle status and checkout.

Conceptual attributes: ID, organisation/property IDs, resident ID, start/effective dates, checkout date when applicable, status, approved references to relevant terms and lifecycle records.

Important: The stay's exact states, transition guards and relationship to room assignment must follow owner discovery (BR-LIF-001, BR-LIF-004). Do not assume a stay begins at application approval or that the bed becomes occupied on the same date as move-in.

7 AssignmentHistory

Records bed/room assignment changes over time.

Purpose: Preserve from/to assignment, effective time, actor, reason when required, and transition references. The current assignment can be queried from a clearly defined current state, but history must not be overwritten.

Open decision: Whether a resident can temporarily have two allocations during a move (BR-LIF-006), and whether temporary absences release inventory (BR-LIF-013).

8 RentalTermsVersion

An effective-dated representation of approved commercial terms for a stay. May contain rent amount and policy identifiers/parameters once the rules have been approved.

Conceptual attributes: stay ID, version number, effective range, base amount/currency, source/agreement reference, approval actor/time, state.

Important: Do not store an assumed rent cycle or preselect anniversary/monthly billing. A policy type/version may be needed, but allowed values are decided after owner confirmation (BR-REN-001, BR-REN-002). Previously issued charges must refer to the exact terms/version used (BR-REN-007).

9 Charge and ChargeLine

A `Charge` is a distinct amount owed, with issue/status information and a stable billing-event identity. `ChargeLine` captures the composition of a charge where separate rent/services/adjustments are needed.

Conceptual attributes: charge ID, organisation/property/stay, source event/reference, charge period if relevant, due date if defined, amount/currency, status, created/issued timestamps, policy/version reference.

Structural invariant — always required: Repeating the same intended billing event must not create a duplicate charge. Implement a stable event key and database-enforced uniqueness after the event identity is specified.

Policy-dependent — blocked until signed off: The content of the event key, the charge period boundaries, the due date and the amount all derive from unresolved rules (BR-REN-001 … BR-REN-006). The structure may be designed; the values may not be computed. A charge may be created manually with an authorised user's supplied amount and reason without any automated calculation. *(C-09)*

Open decisions: Whether the user-facing document is called invoice, bill or charge (BR-REN-013); which fields are legally required; and treatment of taxes and service fees (BR-REN-012).

10 Payment

Represents money reported/received through cash, UPI or bank transfer in P0. Keep payment status and confirmation evidence separate from charge settlement.

Conceptual attributes: payment ID, organisation/property/stay or payer reference, method, amount in exact representation, currency, received/recorded timestamps, external reference when present, status, recorder, confirmation metadata, idempotency/request reference.

Structural invariant — always required: A payment record is never physically deleted after it has affected a statement; corrections are represented by governed reversal/adjustment events.

Policy-dependent — blocked until signed off: Which status transitions exist, what evidence confirms each method, whether cash can be confirmed by the recorder, and how duplicate real-world references are detected (BR-PAY-002 … BR-PAY-005). *(C-09)*

11 PaymentAllocation

Connects an amount of a payment to a charge (or charge line if later justified). One payment can have multiple allocations and one charge can be settled by multiple payments.

Conceptual attributes: allocation ID, organisation ID, payment ID, charge ID, allocated amount, created timestamp, actor, reversal linkage/status if required.

Structural invariant — always required: The sum of valid allocations must be no greater than the allocatable payment amount, enforced inside a transaction with appropriate serialisation.

Policy-dependent — blocked until signed off: Whether pending or unverified payments count toward the allocatable amount (BR-PAY-003); the default allocation order, if any (BR-ALLOC-002 — **no oldest-first assumption**); advance credit behaviour (BR-ALLOC-003); whether allocations can be reassigned and what approval that requires (BR-ALLOC-005). A pending payment must not be reported as settled money unless an approved rule explicitly permits it. *(C-09, C-12)*

12 DepositAccount and DepositEntry

A deposit account is a conceptual ledger for deposit obligations and available refundable balance. Its scope could be per stay or per resident depending on owner policy and legal/accounting advice (BR-DEP-003).

`DepositEntry` records receipt, refund, approved deduction, correction or reversal. Do not model deposit as a negative rent charge.

Conceptual attributes: deposit account ID, organisation/property, resident/stay reference; entry ID, entry type, amount, effective/recorded dates, reason/evidence, actor, approver, reversal reference.

Structural invariant — always required: Deposit activity remains distinguishable from rent collections in every report. A refund or deduction cannot exceed the available deposit balance under the approved rule.

Policy-dependent — blocked until signed off: Fixed vs variable deposit, instalments, permitted deductions, offset against rent, refund deadline, per-resident vs per-stay scope and tax/accounting treatment (BR-DEP-001 … BR-DEP-010). Offsetting a deposit against unpaid rent never happens automatically. *(C-07, C-09)*

13 DocumentMetadata

Stores file metadata and access attributes, not the binary content itself.

Scope note: Binary document upload is **excluded from default P0** (ADR-012, BR-SEC-004). This entity is designed for the conditional case where the owner proves upload is necessary, and for minimal metadata where an approved workflow requires it. It is not built on the assumption that upload will be enabled. *(C-03)*

Conceptual attributes: organisation ID, linked entity type/ID, document category, storage object key, content type/size, uploaded actor/time, access/retention classification, optional expiry date if relevant.

Security: Store binaries in private object storage. Avoid public bucket ACLs. The API must authorise every download; expiring signed URLs should be short-lived and generated only after permission checks.

Open decisions: Which documents are genuinely needed, who may view them, retention schedule, acceptable alternatives and consent/legal basis (BR-SEC-004, BR-SEC-005, BR-OPS-003).

14 AuditEvent

Captures security- and business-relevant actions.

Conceptual attributes: organisation ID when applicable, actor ID, action, target type/ID, timestamp, request/correlation ID, reason and safe/minimised before/after data or event references.

Security: Append-only to application roles where practical. Logs must not contain passwords, access tokens, secret keys, full identity document contents or unnecessary personal data.

15 ImportBatch and ImportRowResult

Support reviewable data migration.

Purpose: Track source file/batch, uploader, validation status, row-level errors, imported/rejected counts, reconciliation totals and approval. Do not silently skip invalid financial rows.

Scope note: These entities are exercised against synthetic fixtures only until the migration gates in `08_TESTING_AND_MIGRATION_PLAN.md` §9 have passed. The existence of import tooling is not evidence that real data is safe to import. *(C-10)*

16 Future entities — not P0

Application/Enquiry, BookingHold, Complaint, MaintenanceTask, Expense, MessageDelivery, ResidentAccountLink, Subscription, PlanEntitlement, ProviderTransaction, BankStatementLine and AccountingExport may be added as separate modules when prioritised. Do not add all of these before the first workflow is validated.

Financial transaction model

1 Source records, not an editable balance

The source of truth is the set of valid charges/adjustments, payment records, payment allocations and deposit entries. A resident's outstanding balance is a derived value using an agreed inclusion/cutoff rule, not a free-standing field that staff edit.

A statement may present:

- Charges and their periods/due dates, as defined by approved policy.
- Credits, adjustments and reversals.
- Payments recorded and their status.
- Allocations that apply eligible payments to charges.
- Unallocated/advance receipts, if supported.
- Deposit activity in a separate section.

The exact accounting presentation and tax labels require owner/accountant review (BR-REN-012, BR-REN-013, BR-DEP-003).

2 Money representation

Decided as proposed in ADR-008. INR amounts are stored as integer paise in a conceptual field `amount_minor`, held in a PostgreSQL `BIGINT` column. The API sends and returns `amountMinor` as a base-10 integer string together with an explicit `currency`, for example `{"amountMinor":"123450","currency":"INR"}` for ₹1,234.50. *(C-01)*

Rules that follow from this:

- Binary floating point is never the source of financial arithmetic, in the database, in TypeScript, in JSON or in the browser.
- The amount string is validated for format and supported range before conversion to the database integer. Signs, decimal points and scientific notation are rejected where the endpoint expects a positive amount.
- Conversion happens once, at the API boundary. Display formatting to rupees happens only at the UI edge.
- Currency is explicit in the model even though INR may be the only supported currency in the first pilot, so the assumption does not become invisible.

Percentage and tax rounding rules are policy-specific and remain **unresolved** until the corresponding features are approved (BR-REN-012).

3 Payment allocation invariants

For each payment `p`:

```
valid allocations(p) <= allocatable amount(p)
```

The exact amount available for allocation must account for approved payment status, reversals and refunds. An unverified record does not reduce an outstanding balance unless the owner approves a precisely defined provisional reporting behaviour (BR-PAY-003).

For each charge `c`, the settled amount is derived from valid allocations/credits under the defined rules. The outstanding amount cannot be negative unless explicit credit/overpayment semantics have been approved (BR-ALLOC-003).

A payment allocation and the associated balance-changing operations must occur in one database transaction. A concurrent allocation must lock or otherwise serialise the relevant records so two requests cannot consume the same available amount.

4 Correction and reversal model

Do not use physical deletion as the correction workflow for issued charges, receipts, allocations or deposit entries. Use linked adjustment/reversal records, with a reason, actor, timestamp and approval where required. Whether a full payment can be reversed, refunded, or reallocated is a business-rule decision (BR-PAY-009, BR-ALLOC-005); the model must preserve enough history to represent those decisions.

5 Idempotency

- Every charge-generation event needs a stable key derived from the approved business event/policy period, not from an arbitrary retry timestamp. The composition of that key is blocked until BR-REN-001 and BR-REN-002 are approved. *(C-09)*
- Financial write endpoints accept an idempotency key or an equivalent client-operation identifier.
- Store the key, authenticated organisation/actor scope, request fingerprint and resulting resource/reference.
- Reuse of the same key with the same request returns the original result; reuse with a materially different request returns a conflict.
- Idempotency must be enforced at the persistence boundary, not only by an in-memory cache.

Request idempotency is a structural invariant and is not a business policy. It prevents a client retry from duplicating an action. It is separate from duplicate detection based on a real-world payment reference such as a UPI reference or bank UTR, whose rules are unresolved (BR-PAY-005). Building the first does not deliver the second. See ADR-009.

Resident lifecycle concept

The eventual lifecycle may contain enquiry → application → approval → move-in/stay active → room change/renewal → notice → operational checkout → financial settlement. This is a conceptual map only. Not all states are P0; actual states and allowed transitions must be based on owner workflow and agreement terms (BR-LIF-001).

Requirements for the model:

- Keep application/booking state separate from an active resident stay.
- Keep resident identity separate from a stay.
- Treat lifecycle changes as explicit events/state transitions with actor and timestamp.
- Define what happens to charges and bed assignment when a transition occurs.
- **Operational checkout and financial settlement are separate states and separate events.** Recording a confirmed departure does not settle money, does not net a deposit against dues, and does not by itself

release a bed. Bed release occurs on the owner-approved operational effective date (BR-LIF-004, BR-LIF-011), which must not be inferred from the checkout event. Financial settlement is a distinct, separately approved workflow (BR-REN-005, BR-DEP-005, BR-DEP-006, BR-DEP-008). *(C-07)*

Bed-assignment constraints

The data model should support a database-enforced rule that one allocatable unit cannot have conflicting active allocations. Implementation choice may use a single current-allocation record, effective-dated assignment intervals with an exclusion constraint, or a carefully designed partial unique constraint after the state model is approved.

Do not choose a specific SQL constraint until the following are confirmed:

- Which states reserve inventory versus occupy it (BR-LIF-004).
- Whether future reservations may overlap at their boundary timestamps.
- Whether one stay may have overlapping assignments during a transfer (BR-LIF-006).
- Whether a resident can temporarily occupy an extra bed.
- Whether the unit being allocated is always a bed, or can be an entire room/private unit (BR-003).

A frontend "bed available" check is not sufficient because concurrent requests can race.

Organisation isolation at the data layer

Proposed pattern, per ADR-007:

- Organisation-owned entities carry an organisation relationship, directly or via a parent with a verified ownership path.
- Queries scope by the authenticated membership resolved server-side. A client-supplied organisation identifier is never trusted as authority, in any position in the request. *(C-05)*
- Repository and service APIs require organisation scope for organisation-owned queries; unscoped `findById` patterns are not used for protected resources.
- Cross-organisation references are prevented by application checks and, where suitable, composite foreign keys including organisation ID.
- Organisation-scoped unique constraints are used where uniqueness is per-customer rather than global.
- PostgreSQL row-level security remains an **OPEN** defence-in-depth option. If adopted, it requires a separately tested decision covering connection-pool and session context handling, transaction boundaries, migration roles and a safe bypass policy for controlled maintenance jobs. Do not assume RLS is active merely because a policy was written, and do not treat it as a substitute for application authorisation.
- Tests must attempt cross-tenant reads and writes at every resource family, including lists, searches, reports, exports, imports, background jobs and file access.

The approved approach is recorded in `10_ARCHITECTURE_APPROVAL_REGISTER.md` under ADR-007. *(C-13)*

Conceptual integrity constraints to consider

These are candidates for schema-level protection after requirements review:

- Unique membership per user/organisation (or a clearly defined equivalent).
- Unique room/bed labels within their required scope.
- Foreign keys connect a resident stay, room/bed, terms, charge, payment and organisation consistently.

- A charge's organisation must match the organisation of its stay and referenced terms.
- An allocation's payment, charge and organisation must be mutually consistent.
- Allocation totals cannot exceed the allowed payment amount; because this is an aggregate rule, enforce through transactions and appropriate locking/constraints.
- Charge-generation event identity is unique within its intended scope.
- A bed cannot have conflicting active assignment intervals.
- Positive/non-negative amount checks according to event type; do not overload sign conventions without documenting them.
- Reversal entries reference the original entry and cannot be reversed inconsistently more than once under the chosen rules.
- Audit events are not writable/deletable by ordinary application identities.

Queries and reporting contract

The team must define, before coding reports (BR-REP-001 … BR-REP-005):

- Which payment statuses count as collected.
- Whether reports are based on received date, recorded date, settlement date or another date.
- What cutoff date applies to a statement.
- How credits/unallocated payments appear.
- Whether deposits are excluded from rent collection totals.
- How reversed/adjusted charges and payments appear.
- How opening balances are represented without manufacturing false historical invoices.

These definitions belong in the business rules and API/report requirements, not as ad hoc frontend arithmetic.

Decisions required before final schema

The conceptual model is not promoted to a database schema until these are resolved. Schema implementation begins only after this conceptual model and the business rules for the enabled features are approved; it is explicitly out of scope for Milestone 0. Migrations are versioned, reviewed in pull requests and tested against a clean PostgreSQL database in CI, per ADR-003 and ADR-010. *(C-06, C-10)*

1. Confirm billing period and due-date rule (BR-REN-001, BR-REN-002).
2. Confirm proration and effective-date behaviour (BR-REN-004, BR-REN-005).
3. Confirm payment verification and allocation policy (BR-PAY-003, BR-ALLOC-001 … BR-ALLOC-005).
4. Confirm deposit scope and settlement rules (BR-DEP-001 … BR-DEP-010).
5. Confirm lifecycle/bed-state model and allowed overlap (BR-LIF-001, BR-LIF-004, BR-LIF-006).
6. Confirm minimum resident data and document handling (BR-LIF-008, BR-SEC-004).
7. Confirm opening-balance representation and migration cutover date (BR-OPEN-001 … BR-OPEN-005).
8. Confirm roles/permissions and sensitive-action approvals (BR-SEC-002, BR-SEC-003).
9. Confirm reporting cutoffs and definitions (BR-REP-001 … BR-REP-005).
10. Approve the database constraint approach, including whether row-level security is adopted (ADR-007).

Money representation is no longer on this list; it is decided as proposed in ADR-008 and stated in §4.2. *(C-01)*

APPENDIX C

API Catalogue and Contracts

The proposed endpoint surface with its permissions, validation rules and error contract, including the endpoints that must refuse to act until a business rule is approved.

> **PROVENANCE**
>
> Reproduced from `04_API_SPECIFICATION.md` (requirements package v0.2). No endpoint in this catalogue exists in the repository; the API currently serves two health routes only.

API principles

- All protected endpoints require authentication and server-side authorisation.
- Organisation context is derived from the authenticated session and verified membership, then checked against the target object. A client-supplied organisation identifier is not an authorisation mechanism in any position — path, query, body or header. *(C-05)*
- Use request/response validation at the API boundary and repeat important invariants in the domain/persistence layer.
- Use consistent error responses and avoid disclosing whether a foreign organisation's record exists.
- Use database transactions for operations that must succeed or fail together.
- Use idempotency protection for financial write operations and retryable bulk imports.
- **Money format, per ADR-008:** every request and response carries `amountMinor` as a base-10 integer string of paise together with an explicit `currency`, for example `{"amountMinor":"123450","currency":"INR"}` for ₹1,234.50. A JSON number is never accepted as an authoritative monetary amount. The string is validated for format and supported range before conversion to the database `BIGINT`. *(C-01)*
- Use ISO 8601 timestamps with explicit offset/UTC representation. Date-only business dates must be separate from timestamps.
- Apply pagination, bounded page sizes, explicit sorting and filter validation to lists.
- **An operation whose governing business rule is unresolved returns `422 RULE_NOT_CONFIGURED`, names the unresolved rule, and writes nothing.** It never falls back to a default, and it never partially commits. *(C-09, C-12)*
- API schema/OpenAPI should be generated from reviewed DTO/contracts after the API is approved; do not let generated framework defaults define business policy.

1 Access notation

The **Access** column in every table below names the **permission identifier** required, not a role. Permissions are granted to roles through the baseline matrix in `10_ARCHITECTURE_APPROVAL_REGISTER.md` §2; the identifier list is in `05_SECURITY_ARCHITECTURE.md` §4. Server-side checks test the permission, never a role-name string. A role gaining a permission is an owner decision recorded in that matrix, not an API change. *(C-04, C-15)*

Where an operation is marked **Blocked**, the endpoint exists in the contract but is disabled until the named business rules are signed off; calling it returns `422 RULE_NOT_CONFIGURED`.

Authentication and organisation access

Session design, per ADR-004 and ADR-005: browser sessions are server-side and revocable, carried in an opaque cookie marked `HttpOnly`, `Secure` in deployed environments, with an appropriate `SameSite` policy and a narrow `Path` /host scope. CSRF protection applies to every state-changing request. The session is rotated after authentication and after any privilege change, and invalidated on account disablement, membership revocation, role change and password reset. Long-lived bearer tokens are not stored in browser storage. *(C-02)*

Still unresolved and required before any shared environment is exposed: idle and absolute session expiry values, password policy, login and recovery rate-limit thresholds, whether MFA is required for owner accounts, and the verified recovery channel or approved owner-assisted process (BR-SEC-006, ADR-005 §3 of document 10). These values are not invented in code.

METHOD/PATH	PURPOSE	ACCESS	NOTES
`POST /api/v1/auth/login`	Authenticate staff user	Public endpoint, rate-limited	Avoid account-enumeration responses. Establishes the server-side session and the active organisation context.
`POST /api/v1/auth/logout`	End and revoke the current session	Authenticated	Invalidates the server-side session record, not only the cookie.
`GET /api/v1/auth/me`	Return current user, active organisation and effective permissions	Authenticated	Returns only memberships available to the user.
`POST /api/v1/auth/recovery`	Begin account recovery	Public endpoint, rate-limited	**Blocked** until the recovery channel is approved (BR-SEC-006, ADR-005). Single-use, short-lived token; only a hash of the token is stored; never security questions.
`GET /api/v1/organisations/current`	Read the active organisation context	Authenticated member	Context comes from the session, not from the request.
`POST /api/v1/organisations/{organizationId}/switch`	Change the active organisation context	Authenticated member of the target organisation	Re-confirms membership server-side and refreshes session context. Membership failure returns `404`, not `403`, so organisation existence is not probeable. *(C-05)*
`GET /api/v1/organisations/current/members`	List staff memberships	`member.manage`	P0 may use controlled/manual provisioning; invitation flow can be staged.

METHOD/PATH	PURPOSE	ACCESS	NOTES
`POST /api/v1/organisations/current/members`	Invite/enable member	`member.manage`	No public organisation creation in P0.
`PATCH /api/v1/organisations/current/members/{membershipId}`	Change role/status	`member.manage`	Sensitive action; audited; triggers session invalidation per ADR-004.

Ordinary resource requests use `/api/v1/organisations/current/...` or resource routes whose target ownership is verified server-side against the session's active organisation. Both forms are acceptable; neither accepts a client-supplied organisation identifier as authority. *(C-05)*

Do not accept a role value from a user and trust it without checking the actor's permission to grant that role. A staff member must not be able to elevate their own access or delegate a permission they do not hold.

Property and inventory resources

METHOD/PATH	PURPOSE	ACCESS	VALIDATION/BEHAVIOUR
`GET /api/v1/properties`	List properties in current organisation	`inventory.read`	P0 likely returns one property.
`POST /api/v1/properties`	Create property	`property.manage`	Validate required fields; creation audit event.
`GET /api/v1/properties/{propertyId}`	Read property	`inventory.read`	Verify organisation ownership.
`PATCH /api/v1/properties/{propertyId}`	Update property	`property.manage`	Field-level permission may be required.
`GET /api/v1/properties/{propertyId}/rooms`	List rooms	`inventory.read`	Paginated/filterable.
`POST /api/v1/properties/{propertyId}/rooms`	Create room	`inventory.write`	Validate labels/types and property ownership.
`PATCH /api/v1/rooms/{roomId}`	Update room	`inventory.write`	Reject invalid changes when active assignment dependencies exist.
`GET /api/v1/rooms/{roomId}/beds`	List beds and status	`inventory.read`	Status derived from the approved allocation model (BR-LIF-004).
`POST /api/v1/rooms/{roomId}/beds`	Create allocatable bed/unit	`inventory.write`	Reject duplicate label within approved scope.
`PATCH /api/v1/beds/{bedId}`	Update bed metadata/status	`inventory.write`	Cannot mark unavailable/removed if a conflicting active allocation exists without an approved transition.
`GET /api/v1/beds/availability`	Query availability	`inventory.read`	Requires defined date/time/state semantics before reservation support (BR-LIF-003, BR-LIF-004). P0 may expose current status only.

Resident and stay resources

METHOD/PATH	PURPOSE	ACCESS	VALIDATION/BEHAVIOUR
`GET /api/v1/residents`	Search/list residents	`resident.read`	Minimise fields; never return document content or full sensitive identifiers.
`POST /api/v1/residents`	Create resident	`resident.write`	Validate required fields per owner-approved policy (BR-LIF-008); check duplicates without exposing unrelated tenant data.
`GET /api/v1/residents/{residentId}`	Read resident	`resident.read`	Verify organisation ownership and scope.
`PATCH /api/v1/residents/{residentId}`	Update resident details	`resident.write`	Audit sensitive changes; do not overwrite lifecycle history.
`GET /api/v1/residents/{residentId}/stays`	List stays	`resident.read`	Return authorised stays only.
`POST /api/v1/residents/{residentId}/stays`	Create stay record	`stay.manage`	Validate property, resident, required approved terms and state transition.
`GET /api/v1/stays/{stayId}`	Read stay	`stay.manage` or `resident.read`	Includes current approved terms and assignment references, not unrestricted documents.
`POST /api/v1/stays/{stayId}/move-in`	Activate/record move-in	`stay.manage`	Idempotency key; transaction covers state and assignment consistency. Required fields remain open (BR-LIF-005).
`POST /api/v1/stays/{stayId}/room-change`	Change allocation	`stay.manage` + `assignment.manage`	Atomic state change; conflict detection; history entry. Whether charges change is unresolved (BR-LIF-007); no charge is altered by this operation until that rule is approved.
`POST /api/v1/stays/{stayId}/notice`	Record notice	`stay.manage`	P0 only if required; notice fields and period rules unresolved (BR-LIF-010).
`POST /api/v1/stays/{stayId}/checkout`	Record **operational** checkout	`stay.manage`	Records a confirmed departure and transitions the stay. **Performs no financial action whatsoever** — no final charge, no deposit net-off, no refund. Bed release occurs only on the approved operational effective date and is **Blocked** until BR-LIF-004 and BR-LIF-011 are signed off. *(C-07)*

State-changing endpoints express business actions rather than allowing arbitrary client edits of `status` fields. The API validates legal state transitions centrally.

Operational checkout and financial settlement are different operations with different permissions. Settlement lives in §7. No response from the checkout endpoint implies, reports or performs a settlement. *(C-07)*

Rental terms and billing resources

METHOD/PATH	PURPOSE	ACCESS	VALIDATION/BEHAVIOUR
`GET /api/v1/stays/{stayId}/rental-terms`	Read term history	`finance.read`	Return effective dates and version references.
`POST /api/v1/stays/{stayId}/rental-terms`	Create term version	`rental_terms.manage`	Reject if mandatory policy fields are unresolved. Never mutate old terms in place (BR-REN-007).
`GET /api/v1/charges`	Search charges	`finance.read`	Filter by resident/stay/status/due date under approved definitions.
`POST /api/v1/charges`	Create an approved manual charge or adjustment	`charge.create`	Require a reason/source; validate allowed charge type; idempotency required. Amount is supplied by the authorised user; nothing is calculated. Automated rent generation uses the separate operation below.
`POST /api/v1/billing-runs`	Generate charges for defined billing events	`charge.create`	**Blocked** until BR-REN-001 … BR-REN-006 are signed off. Returns `422 RULE_NOT_CONFIGURED` naming the unresolved rules. When enabled, must be idempotent on the approved event key and produce a result summary. *(C-09)*
`GET /api/v1/charges/{chargeId}`	Read charge and allocations	`finance.read`	Organisation ownership verified.
`POST /api/v1/charges/{chargeId}/adjustments`	Add credit/adjustment	`charge.adjust`	Original charge retained; reason and actor required. Whether second-person approval is required is unresolved (BR-REN-011).
`GET /api/v1/residents/{residentId}/statement`	Resident account statement	`finance.read`	**Blocked** in part: requires the statement cutoff and payment-status inclusion rules (BR-REP-001, BR-PAY-003). Until approved, the statement lists records without presenting an authoritative outstanding total.

There is no `PATCH` on a posted charge's amount. Corrections use the adjustment or reversal action.

Payments and allocations

METHOD/PATH	PURPOSE	ACCESS	VALIDATION/BEHAVIOUR
`GET /api/v1/payments`	Search/list payment records	`finance.read`	Filter on method/status/recorded or received date, each explicitly named.

METHOD/PATH	PURPOSE	ACCESS	VALIDATION/BEHAVIOUR
`POST /api/v1/payments`	Record cash/UPI/bank-transfer receipt	`payment.record`	Idempotency key required; `amountMinor` positive; method/reference validated. Default status is **Blocked** until BR-PAY-003 is signed off; until then no status may be labelled confirmed.
`GET /api/v1/payments/{paymentId}`	Read payment and allocations	`finance.read`	Hide sensitive reference details where permission requires.
`POST /api/v1/payments/{paymentId}/confirm`	Confirm a pending receipt	`payment.confirm`	**Blocked** until BR-PAY-003 and BR-PAY-002 are signed off. Requires evidence and actor metadata per the approved policy; audited.
`POST /api/v1/payments/{paymentId}/allocations`	Allocate part of a payment to a charge	`payment.allocate`	Amount positive; charge and payment in the same organisation; concurrent allocation serialised; transaction required. **The target charge must be supplied explicitly** — no default ordering is applied while BR-ALLOC-002 is unresolved. *(C-09, C-12)*
`POST /api/v1/allocations/{allocationId}/reverse`	Reverse or reassign an allocation	`allocation.reverse`	**Blocked** until BR-ALLOC-005 is signed off. Reason and approval required; history never mutated silently.
`POST /api/v1/payments/{paymentId}/refunds`	Record a payment refund	`payment.refund`	**Blocked** until BR-PAY-009 is signed off. Requires original-payment linkage, available refundable amount and approval policy. Never creates an uncontrolled negative payment.
`GET /api/v1/receipts/{receiptId}`	View/download receipt	`finance.read`	Stable reference; access checked on every retrieval. Receipt timing and wording are unresolved (BR-PAY-004).

Transaction boundary: record payment and allocate

If the UI records a payment and immediately allocates part of it, the contract must state whether that is one API command. Whichever contract is selected, the backend guarantees atomicity: if an allocation fails validation or concurrency protection, no partial payment/allocation state is committed unintentionally. If unallocated payments are permitted — which is unresolved (BR-ALLOC-004) — payment creation may commit without allocations, and the product must show the resulting unallocated state.

Idempotency contract

For a financial `POST`:

1. Client sends `Idempotency-Key` with a high-entropy operation identifier.
2. Server binds the key to the authenticated organisation/user scope and a request fingerprint.

3. First valid request stores the key and outcome atomically with the operation.
4. Retry with identical key and equivalent request returns the same logical resource/result.
5. Same key with a different payload returns `409 IDEMPOTENCY_KEY_REUSED`.
6. Concurrent identical requests cannot create duplicates; this requires a database uniqueness guard and a transactional outcome.

This is request idempotency and it is an architecture invariant, not a business policy. It prevents a client retry from duplicating an action. It does **not** implement duplicate detection using a real-world payment reference such as a UPI reference or bank UTR, whose rules are unresolved (BR-PAY-005). Shipping the first does not deliver the second, and neither substitutes for the other. See ADR-009.

Deposits and settlement

METHOD/PATH	PURPOSE	ACCESS	VALIDATION/BEHAVIOUR
`GET /api/v1/stays/{stayId}/deposit`	Read deposit ledger/balance	`deposit.read`	Derived only from verified entries. Never included in rent statement totals.
`POST /api/v1/stays/{stayId}/deposit/receipts`	Record deposit received	`deposit.receive`	Validate positive `amountMinor` and payment evidence. Partial-deposit handling is unresolved (BR-DEP-002).
`POST /api/v1/stays/{stayId}/deposit/deductions`	Record approved deduction	`deposit.adjust`	**Blocked** until BR-DEP-005, BR-DEP-009 and BR-DEP-010 are signed off.
`POST /api/v1/stays/{stayId}/deposit/refunds`	Record deposit refund	`deposit.refund`	**Blocked** until BR-DEP-004, BR-DEP-008 and BR-DEP-009 are signed off. Validate available balance; no automatic deduction inference.
`POST /api/v1/stays/{stayId}/settlements/preview`	Preview proposed final settlement	`settlement.preview`	**Blocked.** Computes only from approved rules; every unresolved policy produces `422 RULE_NOT_CONFIGURED` naming the rule. Never nets a deposit against dues unless BR-DEP-006 is signed off.
`POST /api/v1/stays/{stayId}/settlements`	Confirm settlement	`settlement.approve`	**Blocked** until BR-REN-005, BR-LIF-010, BR-LIF-012, BR-DEP-005, BR-DEP-006 and BR-DEP-008 are signed off and approved expected-result tests exist. Persists the settlement decision; audited; idempotency required. *(C-07)*

A settlement is never a side effect of a checkout. The two operations have separate permissions, separate audit events and separate approvals. *(C-07)*

Reports, audit and migration

METHOD/PATH	PURPOSE	ACCESS	VALIDATION/BEHAVIOUR
`GET /api/v1/reports/outstanding`	Outstanding report	`report.read`	Date/cutoff/status semantics explicit in OpenAPI and on the report face; derives from source records. Requires BR-REP-001.
`GET /api/v1/reports/collections`	Collection report	`report.read`	Basis date unresolved (BR-REP-002); the report states which date it uses.
`GET /api/v1/reports/occupancy`	Current inventory status	`report.read`	Statuses follow approved lifecycle semantics (BR-REP-004).
`GET /api/v1/reports/deposits`	Deposit report	`report.read`	Separate from rent collection by construction (BR-REP-003).
`GET /api/v1/audit-events`	Search permitted audit records	`audit.read`	Filtered, paginated, sensitive payloads minimised.
`POST /api/v1/import-batches`	Create import/validation batch	`migration.manage`	File size/type/format controls; malware scanning as appropriate; never commits on upload.
`POST /api/v1/import-batches/{id}/validate`	Validate/preview import	`migration.manage`	Return row-level errors and aggregate reconciliation totals.
`POST /api/v1/import-batches/{id}/commit`	Commit approved import	`migration.approve`	Idempotent, guarded, auditable and transactionally grouped where feasible. Real-data commit additionally requires the gates in `08_TESTING_AND_MIGRATION_PLAN.md` §9. *(C-10)*
`GET /api/v1/import-batches/{id}/report`	View reconciliation report	`report.read` or `migration.manage`	No hidden rejected rows or unreported value differences.
`GET /api/v1/exports/residents`	Controlled resident export	`data.export`	Audited, restricted columns, bounded volume, protected file.
`GET /api/v1/exports/financial`	Controlled financial export	`data.export` + `finance.read`	Audited; export values reconcile to the report definition.

Documents — not in the default P0 contract

Binary document upload and download are excluded from default P0 (ADR-012, BR-SEC-004). No upload, download-link or delete route is part of the P0 API contract, and none is implemented or left reachable. *(C-03)*

This section is retained as a **conditional future design note**. If the owner proves that document handling is necessary for safe initial operation, the following must be approved *before* any route is added: document categories and purpose, permitted viewer roles, lawful basis and resident notice, accepted file types and size limits, malware-scanning process, private storage configuration, signed-link expiry, audit events and the retention and deletion schedule (BR-SEC-004, BR-SEC-005, BR-OPS-003).

The conditional design, if later approved, would use: a controlled upload initiation route, a metadata read route, a short-lived authorised download-link route, and a delete route permitted only where retention and legal rules allow. No generic public file URL is used in any case. A user may be permitted to read a resident

profile without being permitted to read that resident's documents; `resident.read` and `document.read` are separate permissions and one never implies the other.

Error response contract

Use a predictable envelope, for example:

``` `json { "error": { "code": "VALIDATION_FAILED", "message": "The request could not be accepted.", "requestId": "correlation-id", "details": [ { "field": "amountMinor", "code": "MUST_BE_POSITIVE" } ] } } `
```

This is an example response shape, not production code. Do not expose stack traces, SQL, secrets, tokens or another organisation's resource details.

Recommended HTTP meanings:

- `400 BAD_REQUEST`: malformed request/unsupported shape.
- `401 UNAUTHENTICATED`: no valid authentication.
- `403 FORBIDDEN`: authenticated but lacks permission; where resource existence would leak, use a consistent non-disclosing response.
- `404 NOT_FOUND`: resource absent or intentionally concealed for isolation, including an organisation the actor is not a member of.
- `409 CONFLICT`: duplicate event, stale version, idempotency conflict or state/concurrency conflict.
- `409 IDEMPOTENCY_KEY_REUSED`: same idempotency key with a materially different payload.
- `422 UNPROCESSABLE_ENTITY`: valid syntax but invalid business rule, incompatible state or amount constraint.
- `422 RULE_NOT_CONFIGURED`: the operation depends on a business rule that has not been approved. The response names the unresolved rule identifiers. No record is written and no default is applied. *(C-09, C-12)*
- `429 TOO_MANY_REQUESTS`: rate limit exceeded.
- `500 INTERNAL_ERROR`: generic public message with logged request ID; no internal details.
- `503 SERVICE_UNAVAILABLE`: temporary dependency/maintenance failure where retry may be appropriate.

Stable error codes must be defined in the OpenAPI contract and covered by tests.

Cross-cutting validation rules

- Reject unknown fields for sensitive write DTOs where feasible; do not mass-assign client objects to persistence entities.
- Validate IDs, dates, status transitions, amount strings and ranges, text lengths and permitted enums.
- Check entity ownership before joining/updating linked records.
- Validate effective-date ranges and prevent silently overlapping terms where overlap is not allowed.
- Validate payment/allocation/refund totals inside transaction boundaries, not just before transaction start.
- Treat frontend validation as usability support; backend validation is authoritative.
- Avoid returning document identifiers, bank references or resident details in logs/errors without need.
- Use pagination and sensible query limits; protect search/export endpoints from abuse.

Open API decisions before implementation

1. Exact session idle and absolute expiry values, password policy, rate-limit thresholds, owner MFA decision and recovery channel (ADR-005, BR-SEC-006). *(The session mechanism itself is decided — ADR-004.)*
2. Exact role-to-permission assignments in the baseline matrix (ADR-006, BR-SEC-002, BR-SEC-003).
3. Payment statuses and verification transitions (BR-PAY-003).
4. Allocation order, unallocated credit and correction semantics (BR-ALLOC-001 … BR-ALLOC-005).
5. Billing event identity and charge issue/status lifecycle (BR-REN-001, BR-REN-002).
6. Checkout operational transition and bed-release effective date (BR-LIF-004, BR-LIF-011).
7. Settlement actions and approvals (BR-DEP-005, BR-DEP-006, BR-DEP-008, BR-DEP-009).
8. Import format, maximum size, commit granularity and rollback behaviour (BR-OPEN-001 … BR-OPEN-005).
9. Rate limits, pagination defaults, API versioning and observability contract.
10. Whether document handling enters P0 at all (BR-SEC-004, ADR-012).

Resolved since version 0.1 and therefore removed from this list: money representation and JSON format, now ADR-008 *(C-01)*; the organisation-context selection mechanism, now one server-side active context with an explicit switch operation *(C-05)*; the browser session mechanism, now ADR-004 *(C-02)*.

APPENDIX D

Acceptance Criteria and Traceability

The test strategy, the financial scenario suite, the concurrency and security tests, and the migration and reconciliation contract that gate the pilot.

> **PROVENANCE**
>
> Reproduced from `08_TESTING_AND_MIGRATION_PLAN.md` (requirements package v0.2). Class (a) tests assert architecture invariants and are always required; class (b) tests cannot assert an expected value until the owner signs off the rule behind it.

Test strategy

Testing must protect business invariants and not merely confirm that pages render. The priority order is:

1. Financial correctness and transaction integrity.
2. Cross-organisation isolation and role permissions.
3. Bed/resident lifecycle consistency.
4. Migration and reconciliation.
5. Backup and recovery.
6. Workflow usability and responsive behaviour.
7. Ordinary page/component behaviour and edge-case validation.

Use a testing pyramid: unit tests for pure domain calculations and validation, integration tests against a real PostgreSQL test database for transactions and constraints, API tests for authorisation and contracts, and end-to-end tests for complete owner and staff workflows. Mocking a repository is not evidence of database isolation or concurrency correctness.

1 Test classification — the rule that governs this whole document

Every test in this plan is one of two kinds. The distinction determines whether the test may exist before the owner has decided anything. *(C-12)*

CLASS	DEFINITION	MAY BE WRITTEN BEFORE OWNER SIGN-OFF?	EXPECTED RESULT COMES FROM
(a) Architecture invariant	A correctness property that holds regardless of which commercial policy the hostel chooses. No duplicate on retry; no over-allocation beyond the approved allocatable amount; traceable corrections; no cross-tenant access; atomic commit; exact money arithmetic.	**Yes.** These are required and are a release gate.	`02_BUSINESS_RULES_REGISTER.md` §9

CLASS	DEFINITION	MAY BE WRITTEN BEFORE OWNER SIGN-OFF?	EXPECTED RESULT COMES FROM
(b) Owner-policy example	A test whose expected output depends on a commercial rule the owner chooses — a prorated first charge, an allocation order, a late fee, a deposit net-off, a refund deadline.	**No.** The scenario may be *listed*, but no expected value may be asserted.	A signed-off answer in `11_OWNER_BUSINESS_RULES_SIGNOFF_CHECKLIST.md`

Two consequences, both non-negotiable:

- **A class (b) test with an invented expected value is worse than no test**, because it converts a guess into an enforced requirement and makes the guess look reviewed. A test suite is a specification; writing one for an undecided rule decides the rule by default.
- **A class (b) test that has not been signed off is reported as pending, never as passing or failing.** A suite summary that counts pending policy tests as green misrepresents readiness.

Specifically, do **not** assert a default allocation order, a pending-payment treatment, a deposit net-off or a refund timing until each is explicitly approved. For a policy-dependent operation whose rule is unapproved, the test asserts that the system returns `422 RULE_NOT_CONFIGURED` and writes nothing — that assertion is itself a class (a) invariant. *(C-09, C-12)*

Unit tests

Financial domain

- **Exact amount parsing, serialisation and formatting per ADR-008:** `amountMinor` accepted only as a base-10 integer string; rejection of decimal points, signs where a positive amount is required, scientific notation, leading-zero ambiguity, empty strings and values outside the supported range; round-trip integrity between the string, the in-memory value and the `BIGINT` column; confirmation that no binary floating-point type appears anywhere in the money path. *(C-01)* — class (a)
- Amount bounds and allowed precision. — class (a)
- Calculation of balance from charges, credits, allocations and reversals under the approved rules. — class (a) for the arithmetic; class (b) for any inclusion rule depending on BR-REP-001 or BR-PAY-003
- Effective-dated rental-term selection. — class (a)
- Charge-generation event-key construction. — class (b), blocked by BR-REN-001 and BR-REN-002
- Validation of allocation amount against available amount. — class (a)
- Deposit ledger balance calculations, separate from rent. — class (a)
- Adjustment and reversal eligibility. — class (a) for structure; class (b) for approval thresholds
- Statement cutoff and status inclusion rules. — class (b), blocked by BR-REP-001

No unit test guesses an unresolved rent, proration or late-fee rule. Class (b) tests are derived only from approved examples recorded in document 11.

Lifecycle and inventory

- Allowed and forbidden state transitions. — class (a) for the transition machine; class (b) for which states exist (BR-LIF-001)
- Bed assignment state calculations. — class (a)

- Effective-date boundary rules. — class (b), blocked by BR-LIF-004
- Room change and checkout event validation. — class (a); the bed-release date is class (b), blocked by BR-LIF-004 and BR-LIF-011
- Resident/stay separation and historical record retention. — class (a)

Access and validation

- Permission identifier mapping, table-driven across every permission in `05_SECURITY_ARCHITECTURE.md` §4.1. Adding a permission or a role without adding a test row fails the build. — class (a)
- DTO field validation and rejection of unexpected privileged fields. — class (a)
- Redaction of sensitive values in safe error messages and log events. — class (a)

Database integration tests

Run against PostgreSQL configured similarly to the proposed production deployment, at the same major version. SQLite is not a substitute for relational transaction and constraint evidence.

Integrity tests — all class (a)

- Invalid foreign-key and cross-organisation relationships are rejected.
- Duplicate charge event keys are rejected.
- Duplicate idempotency keys cannot create multiple financial records.
- Allocation totals cannot overspend a payment under concurrent operations.
- Bed assignment constraints reject conflicting active assignments under the approved state model.
- Financial updates roll back completely when any part of a payment, allocation or settlement transaction fails.
- Corrections create traceable reversal or adjustment entries and retain original records.
- Imports are idempotent and have a defined rollback or partial-commit contract.
- A policy-dependent operation whose rule is unresolved writes nothing and returns `422 RULE_NOT_CONFIGURED`.

Isolation tests — all class (a)

- Two organisations have separate seed data and authenticated test users.
- Direct resource access, nested paths, list/search/report queries and linked-record operations do not leak across organisations.
- A client-supplied organisation identifier that disagrees with the session context never widens access. *(C-05)*
- An organisation the actor does not belong to returns `404`, so existence cannot be probed.
- If PostgreSQL row-level security is selected, tests run using the same application database role and connection-pool pattern used in production; a privileged test role must not accidentally bypass the behaviour being tested.
- Background jobs, exports, imports and any file access receive and enforce organisation context.

API tests

For every protected route, test:

- Valid authorised request.
- Missing or invalid authentication.

- Authenticated actor lacking the required permission identifier.
- Object belonging to another organisation.
- Malformed or invalid input, including a JSON-number amount where a string is required.
- Missing required policy configuration, returning `422 RULE_NOT_CONFIGURED` with the unresolved rule named.
- Stale state, version or concurrency conflict.
- Idempotent retry of the same request.
- Idempotency key reused with a different payload, returning `409 IDEMPOTENCY_KEY_REUSED`.
- Generic errors do not leak stack traces, SQL or private record details.
- Pagination and filter parameters are bounded and validated.

Financial endpoints must have explicit test cases for how retries behave after a client timeout where the server may have committed successfully but the client did not receive the response.

The cross-tenant suite is enumerated from the router automatically, so a newly added route cannot be forgotten. A single leak fails the build.

Financial scenario suite

Every scenario below must be **defined**. Whether it can be **asserted** depends on its class. A class (b) row without a signed-off expected result is recorded as pending; it is never counted as passing, and no expected value is invented to make it run. *(C-12)*

ID	SCENARIO	CLASS	REQUIRED ASSERTION	BLOCKED BY
FIN-001	Approved charge is created	(b)	Charge refers to the correct stay and terms; amount and due information match the approved rule.	BR-REN-001, BR-REN-002
FIN-002	Billing request is retried	(a)	No duplicate charge for the same event.	—
FIN-003	Resident makes a partial payment	(a)	Only the validly allocated amount settles the charge; the remaining amount is correct.	—
FIN-004	One payment allocated across multiple charges	(a)	Total allocation never exceeds the allocatable payment amount.	—
FIN-005	Several payments settle one charge	(a)	Combined valid allocations and outstanding balance are correct.	—
FIN-006	Unallocated or unmatched payment is recorded	(b)	Visible as unallocated and not counted as settled rent.	BR-ALLOC-004
FIN-007	Payment pending verification	(b)	Reports apply the approved inclusion rules and do not mislabel it as confirmed collection.	BR-PAY-003
FIN-008	Same request submitted twice after a client timeout	(a)	Only one logical transaction exists.	—
FIN-009	Two concurrent allocations use the same available amount	(a)	At most the valid amount is allocated; the loser receives a conflict response.	—
FIN-010	Payment correction or reversal is made	(a) structure, (b) authority	Original remains traceable; resulting statement reconciles.	BR-PAY-009 for who may do it
FIN-011	Charge adjustment or credit is made	(a) structure, (b) approval	Reason and actor recorded; balance changes according to policy.	BR-REN-011

ID	SCENARIO	CLASS	REQUIRED ASSERTION	BLOCKED BY
FIN-012	Security deposit receipt recorded	(a)	Deposit balance changes; rent collection does not.	—
FIN-013	Deposit refund or deduction attempted above available amount	(a)	Request is rejected.	—
FIN-014	Checkout with outstanding rent and a held deposit	(a)	**Operational checkout changes no financial balance.** No net-off occurs. Settlement is a separate operation that is refused while its rules are unresolved. *(C-07)*	— for the invariant; BR-DEP-006 for any approved net-off behaviour
FIN-015	Statement and aggregate report for the same scope and cutoff	(a) arithmetic, (b) inclusion rule	Totals reconcile under the report contract.	BR-REP-001
FIN-016	Historical charge terms are changed	(a)	An already-issued charge is not silently rewritten.	—
FIN-017	Payment entered with an unknown reference	(b)	Behaviour follows the approved unverified/unmatched policy.	BR-PAY-006, BR-ALLOC-004
FIN-018	Duplicate real-world reference entered for different payment records	(b)	Warning or blocking behaviour follows the approved duplicate policy and never silently merges payments.	BR-PAY-005
FIN-019	Allocation attempted with no target charge while no ordering policy is approved	(a)	The request is refused; no charge is selected automatically. *(C-09, C-12)*	—
FIN-020	Any policy-dependent financial operation invoked while its rule is unresolved	(a)	Returns `422 RULE_NOT_CONFIGURED` naming the rule, writes nothing, and applies no default.	—

Of twenty scenarios, eleven are class (a) and can be built and enforced now. Seven are class (b) and are pending owner sign-off. Two are mixed and can be partially enforced. **A suite summary must report these three counts separately.** A single "18/18 passing" figure that hides seven undecided policies misrepresents readiness to the owner and to the team.

Concurrency tests

Use independent database connections and coordinated parallel requests, not only sequential unit tests. All class (a).

- Two staff allocate the last available amount of the same payment concurrently.
- Two staff assign the same bed to different stays concurrently.
- Two billing jobs generate the same billing event concurrently.
- Two import commits attempt to process the same batch or event concurrently.
- A user changes rental terms while another process attempts to generate a charge using a version.
- A payment correction races with a new allocation.

Expected outcome: invariants remain true; requests are serialised, rejected or retried according to a documented policy; no silent double allocation or double billing occurs.

End-to-end tests

At minimum:

1. Owner signs in, creates and inspects property inventory, adds a resident and records a stay and assignment.
2. A user with the required permissions records the resident's owner-approved rental terms and creates the appropriate charge.
3. A user with `payment.record` and `payment.allocate` records a payment, allocates it to an explicitly chosen charge, views the receipt and verifies the statement.
4. Owner views the outstanding report and confirms the resident and aggregate values match.
5. A deposit is recorded and appears separately from rent collection in every view and export.
6. A room change follows approved state transitions and retains history.
7. An **operational checkout** is recorded; every financial balance is unchanged afterwards and the UI says so. *(C-07)*
8. A user lacking a required permission is blocked even by direct URL and direct API call.
9. Import preview shows invalid rows and totals; an approved import commits; a repeated import does not duplicate.
10. A failed network response during a financial write can be safely retried using the idempotency key.
11. A policy-dependent action is visibly disabled, states which rule is missing, and cannot be triggered by direct URL or API.
12. Responsive layouts let staff complete the key workflows at the agreed desktop and phone viewport sizes.

Security and privacy tests

- Cross-tenant reads and writes via direct ID substitution, filters, nested routes, reports, exports, import rows and any file identifier.
- A client-supplied organisation identifier conflicting with the session context.
- Role escalation and privilege-grant attempts, including attempting to grant a permission the actor does not hold.
- Session revocation and disabled user access; session invalidation on role change, membership revocation and password reset.
- CSRF protection on every state-changing request; CORS and cookie attribute review against ADR-004.
- Rate limits on login, recovery, search, export and high-impact writes.
- If document handling was approved: file upload validation, size and type restrictions, access revocation and expiring-link behaviour. If not approved: confirmation that no upload or download route is reachable. *(C-03)*
- Sensitive-data redaction in logs, errors and audit payloads.
- Backup and object-store access permissions.
- Secret scanning in repository, build artefacts and deployment configuration.
- Dependency and configuration vulnerability checks.

Reconciliation contract

At minimum reconcile:

- Total residents by the agreed current and historical definition.
- Total beds, rooms and active assignments.
- Resident-level opening outstanding balances.
- Sum of opening outstanding balances.

- Deposit liability and balances by resident or stay, and in aggregate.
- Total imported payments by method, status and date scope if included.
- Unallocated and unmatched amounts, separately.
- Any opening adjustment used to represent a verified balance where detailed historical components are unavailable.

Do not create a fake payment or a fake charge to make two totals match. If an opening balance is imported as an opening item, label its provenance and approval clearly and avoid implying unavailable history.

Backup and restore tests

Before go-live:

1. Confirm the automated backup schedule, retention, encryption and access permissions.
2. Take or identify a known backup and recovery point.
3. Restore into an isolated environment that cannot write to production.
4. Verify database migration and version compatibility, and object-storage consistency where applicable.
5. Compare resident, stay and bed counts, charge, payment, allocation and deposit totals, and representative statements, to recorded pre-restore values.
6. Verify audit events and important references survived.
7. Record the full evidence set: backup identifier and timestamp, restore outcome, schema and application version, reconciliation totals, the named verifier, time taken and any limitations. *(C-11)*
8. Repeat after material changes and on a schedule approved by the operator.

Recovery readiness is claimed only from a completed restore with that evidence record. A dashboard reporting a successful backup is not evidence, and a restore without reconciliation totals is not evidence either. See `05_SECURITY_ARCHITECTURE.md` §10 and ADR-016.

RPO and RTO remain **unresolved**. Set them after a conversation about how much data loss and downtime the operator can tolerate (BR-OPS-005), and after the hosting and backup design is selected (ADR-010). Do not publish an availability promise before hosting and recovery capabilities are known.

Defect severity and release policy

- **Critical:** data exposure across organisations, unrecoverable or corrupt financial data, uncontrolled loss of records, major authentication bypass. Blocks go-live.
- **High:** incorrect charge, payment or deposit outcome, duplicate financial record, bed conflict, failed recovery, unauthorised sensitive action, a policy-dependent operation executing on an unapproved default. Blocks go-live.
- **Medium:** significant workflow issue with a safe workaround and no integrity or security impact. Owner accepts with a documented workaround.
- **Low:** cosmetic or minor usability issue that does not impair the core task.

No critical or high defect involving security or financial integrity may be waived to meet a schedule. Release gates `RG-2` and `RG-4` in `01_PRD.md` §9 depend on this policy. *(C-14)*

APPENDIX E

Owner Decision Worksheet

The decisions formatted for a working session with the owner. Deliberately blank: it is filled in during the workshop, not before it.

> **PROVENANCE**
>
> Reproduced from `11_OWNER_BUSINESS_RULES_SIGNOFF_CHECKLIST.md` (requirements package v0.2). Answer fields are intentionally empty. No answer has been pre-filled, suggested or inferred.

Sign-off metadata

- Hostel / property label: _____
- Legal operator / partnership name (as appropriate): _____
- Owner / authorised approver: _____
- Staff consulted: _____
- Workshop date: _____
- Evidence reviewed (registers, agreements, receipts, redacted bank evidence): _____
- Person responsible for follow-up questions: _____

For each line, record **Decision / Evidence / Approved by / Date / Test case ID**. If the answer varies by resident or agreement, document the scope and effective period rather than making one global rule.

The **Test case ID** field is not optional. A policy answer with no corresponding test is a rule nobody will notice the software breaking. See `08_TESTING_AND_MIGRATION_PLAN.md` §1.1 — these answers supply the expected results for every class (b) test. *(C-12)*

Rent terms, billing periods and due dates

- ☐ **BR-REN-001 — Due-date rule:** What determines each resident's due date? Fixed calendar date, admission-related date, agreement-specific date, or another rule? **No default assumed.**
- ☐ **BR-REN-002 — Charge period:** What dates does each charge cover? Are start/end dates inclusive? How are calendar months or other recurring periods represented?
- ☐ **BR-REN-003 — First due date:** At admission, when is the first payment due and what is that payment intended to cover?
- ☐ **BR-REN-004 — Mid-period admission:** Is the first amount full-period, prorated, a fixed agreed amount, or something else? Provide at least two worked examples and an approved expected result for each.
- ☐ **BR-REN-005 — Checkout charge stop:** What is the rent stop date? Is any amount due after physical departure because of notice terms? **Do not infer.**
- ☐ **BR-REN-006 — Calendar edge cases:** If a due date is the 29th, 30th or 31st, what happens in a shorter month? Does policy use a different convention for any resident?
- ☐ **BR-REN-007 — Term changes:** Who may approve rent changes, their effective date, and treatment of already-issued charges? Old charges must not be silently rewritten.
- ☐ **BR-REN-008 — Grace periods and late fees:** Are they used? If so, amount/formula, start point, cap, waiver power and applicable agreement/legal review. Leave disabled unless completely approved.

- ☐ **BR-REN-009 — Weekend and holiday due dates:** If a due date falls on a weekend or public holiday, what applies?
- ☐ **BR-REN-010 — Other charges:** Food, electricity, laundry, damage, amenities or other fees; how measured, priced, approved and billed? Mark each as absent, manual, or in scope.
- ☐ **BR-REN-011 — Discounts, credits and write-offs:** Who can apply them, why, how are they evidenced, and is second-person approval required?
- ☐ **BR-REN-012 — Tax treatment:** Are taxes added to resident charges? **Professional confirmation required** — confirm with the hostel's accountant against the current official position for this specific accommodation arrangement. Do not rely on a summary from a commercial website.
- ☐ **BR-REN-013 — Invoice/statement format:** What documents does the hostel actually issue? Confirm invoice/receipt terminology and any applicable invoicing requirement with its accountant.

Admission, occupancy and checkout

- ☐ **BR-LIF-001 — Admission approval:** Who may approve an applicant and at what point does the stay become active?
- ☐ **BR-LIF-002 — Bed choice:** Is the resident's choice a preference, request, temporary hold or binding allocation? Who has final authority?
- ☐ **BR-LIF-003 — Temporary bed hold:** Is a hold used at all? If so, what duration, what expiry behaviour, and is money collected to create it?
- ☐ **BR-LIF-004 — Bed availability:** At which exact event or date does a bed become unavailable, and become available again? **A bed-release date must never be inferred from a checkout event.**
- ☐ **BR-LIF-005 — Move-in evidence:** Required fields and documents, effective date, deposit status and approval requirements.
- ☐ **BR-LIF-006 — Transfer overlap:** May a resident hold more than one bed during a transfer?
- ☐ **BR-LIF-007 — Room/bed changes:** Who approves a move, when it takes effect, and whether rent or other charges change.
- ☐ **BR-LIF-008 — Mandatory resident fields:** Which fields are genuinely required, for what purpose, and what alternatives exist? Data-minimisation review.
- ☐ **BR-LIF-009 — Required identity documents:** Which IDs or documents are required, if any? **Professional review required** alongside BR-SEC-005.
- ☐ **BR-LIF-010 — Notice requirement:** Notice period, allowed submission channel, effective date and who acknowledges it. If resident or contract categories differ, identify them.
- ☐ **BR-LIF-011 — Formal operational checkout:** Required physical/key handover, inspection, meter/utility confirmation if applicable, and bed release. **This is the operational workflow only.** Answering it does not authorise any financial calculation — that is section E and BR-REN-005. *(C-07)*
- ☐ **BR-LIF-012 — Overstay and early departure:** Rules, if any, and who approves exceptions. Leave any calculation disabled until approved.
- ☐ **BR-LIF-013 — Temporary absence and return:** How is a resident's temporary absence handled, what is recorded, and do charges pause?

Payment entry, verification and receipt

- ☐ **BR-PAY-001 — Methods:** Confirm accepted methods: cash, UPI, bank transfer and any others.
- ☐ **BR-PAY-002 — Evidence per method:** What proves receipt for cash, UPI and bank transfer? What reference and document are required or optional?

- ☐ **BR-PAY-003 — Status model:** Can staff save a payment as reported/pending before independent confirmation? Who may mark it confirmed, and what evidence is required?
- ☐ **BR-PAY-004 — Receipt timing:** When is a receipt issued: at entry, after verification, or after allocation? What wording must distinguish reported from confirmed?
- ☐ **BR-PAY-005 — Duplicate real-world reference:** What fields are unique for each payment method? How should possible duplicates be reviewed without rejecting legitimate similar transactions?
- ☐ **BR-PAY-006 — Unidentified transfer:** Who investigates, how it is held, and what action can resolve or return it?
- ☐ **BR-PAY-007 — Cash reconciliation:** Who counts and compares cash, at what frequency, what records are compared and who signs off discrepancies?
- ☐ **BR-PAY-008 — Transfer reconciliation:** Which bank/UPI source is treated as evidence and who confirms receipt? Do not treat a screenshot alone as authoritative if owner policy says otherwise.
- ☐ **BR-PAY-009 — Corrections and refunds:** Who may correct a mistaken entry, reverse or reassign an allocation, or issue a refund? What reason, evidence and second approval are required?
- ☐ **BR-PAY-010 — Cashbook:** Is cash balance or cashbook tracking needed in the first release?

Payment allocation and opening balances

- ☐ **BR-ALLOC-001 — Allocation choices:** Can a payment be allocated to one charge or several? Can the resident specify the target bill?
- ☐ **BR-ALLOC-002 — Default ordering:** If the payment does not name a charge, which bill (if any) is settled first? **No oldest-first assumption.** If no default is authorised, the system will require explicit staff selection for every allocation.
- ☐ **BR-ALLOC-003 — Advance/overpayment:** Is surplus held as unallocated credit, returned, allocated by choice or handled another way? Who approves?
- ☐ **BR-ALLOC-004 — Unallocated receipts:** Are payments allowed to be recorded before they can be matched? What is shown as outstanding until matching or confirmation?
- ☐ **BR-ALLOC-005 — Reversal:** How is an allocation corrected after an error? Which entries remain visible and who approves?
- ☐ **BR-OPEN-001 — Source of truth:** Identify registers/statements that establish current residents, bed inventory, dues, payments, deposits and credits. Where two disagree, which wins?
- ☐ **BR-OPEN-002 — Opening balance definition:** At what cut-off date are dues, credits, confirmed payments and unconfirmed receipts measured? Prevent double-counting a receipt already reflected in a balance.
- ☐ **BR-OPEN-003 — Per-resident reconciliation:** Approve resident count, current stay/bed assignment, charges/dues, payments/allocations, and deposit balance separately.
- ☐ **BR-OPEN-004 — Exceptions:** List every missing, disputed, conflicting or estimated amount. Decide whether it must be resolved before cutover or can remain explicitly flagged outside the authoritative balance.
- ☐ **BR-OPEN-005 — Sign-off:** Owner signs a reconciliation report showing row counts, total money amounts by category and all approved differences before real-data cutover.

Security deposits, deductions and refunds

- ☐ **BR-DEP-001 — Deposit amount:** Fixed, room-dependent, a multiple of rent, instalments or agreement-specific? No default assumed.

- ☐ **BR-DEP-002 — Collection timing:** Due date, accepted methods, partial collection and receipt requirements.
- ☐ **BR-DEP-003 — Deposit classification:** How is deposit money recorded and accounted for separately from rent, and what statement should the resident and owner see? Confirm with accountant where needed.
- ☐ **BR-DEP-004 — Refundability:** Whether and under what contract or policy conditions it is refundable.
- ☐ **BR-DEP-005 — Permitted deductions:** Enumerate each permitted deduction type, evidence needed, who estimates and approves it, and the resident dispute process. No deductions until approved.
- ☐ **BR-DEP-006 — Offset against rent:** May deposit be applied to unpaid rent or other dues? If yes, under which agreement, legal and accounting basis, and with what approval? **Do not automatically net off.**
- ☐ **BR-DEP-007 — Available balance:** How should partial receipts, deductions, refunds and corrections affect the available deposit balance?
- ☐ **BR-DEP-008 — Refund deadline and process:** Who confirms the amount, who executes the transfer or cash refund, what evidence proves it, and what happens if disputed?
- ☐ **BR-DEP-009 — Approvals:** Identify maker, reviewer, approver, any amount thresholds and whether one person may both initiate and approve.
- ☐ **BR-DEP-010 — Damage documentation:** Inspection record, evidence, resident acknowledgement and dispute process.

Roles, data access and documents

Role names below are the canonical names: **Owner/Partner, Property Manager, Finance Clerk/Cashier, Warden/Front Desk.** The baseline capability matrix they map to is in `10_ARCHITECTURE_APPROVAL_REGISTER.md` §2. *(C-15)*

- ☐ **BR-SEC-001 — Named staff:** Assign each real staff member to a role and identify exceptions.
- ☐ **BR-SEC-002 — Financial visibility:** Which roles may see balances, statements, payment references, collections and exports? Answer for each of the four roles separately.
- ☐ **BR-SEC-003 — Sensitive actions:** Who can create or adjust charges, confirm a payment, correct an allocation, issue a refund, change roles, export data or commit a migration?
- ☐ **BR-SEC-004 — Documents:** Which documents are truly required for the first release, why, on what legal basis and notice, with what alternatives, permitted viewers and retention period? **Default proposal is no binary upload in P0** (ADR-012). Answering "yes" here opens a substantial amount of additional work and risk; answering "no" costs nothing that cannot be added later.
- ☐ **BR-SEC-005 — Identity verification:** Is Aadhaar required at all? Which lawful verification process and alternative IDs are accepted? Obtain professional and local confirmation before building verification functionality.
- ☐ **BR-SEC-006 — Recovery:** Which verified contact channel is available for staff account recovery, or should recovery be owner-assisted? Who confirms a recovery request?

Reports, acceptance and business ownership

- ☐ **BR-REP-001 — Outstanding report:** Define the exact cutoff, included charge/payment statuses and meaning of "outstanding".
- ☐ **BR-REP-002 — Collection report:** Define whether it is based on recorded date, receipt date, confirmation date or another event.
- ☐ **BR-REP-003 — Deposit report:** Define what is included and confirm separation from rent collection.

- ☐ **BR-REP-004 — Occupancy report:** Define active, reserved, unavailable and vacant for the hostel's actual process.
- ☐ **BR-REP-005 — Report priority:** Which reports are essential daily and which monthly?
- ☐ **BR-OPS-001 — Reminders:** Are reminders sent automatically? Which channel, timing, template, and what consent or opt-out applies?
- ☐ **BR-OPS-002 — Expenses:** Are expense records required for the first release?
- ☐ **BR-OPS-003 — Retention:** How long are records and documents retained? **Professional confirmation required.**
- ☐ **BR-OPS-004 — Resident data requests:** How will a resident request correction, deletion or access? **Professional confirmation required.**
- ☐ **BR-OPS-005 — Recovery expectations:** How much data loss and downtime can the business tolerate? This becomes RPO and RTO.
- ☐ **BR-OPS-006 — Discrepancy escalation:** Name who investigates exceptions and the target response time. Do not encode an SLA until agreed.
- ☐ **BR-OPS-007 — Pilot success:** Agree measurable criteria that would let the owner stop duplicating the paper register.
- ☐ **BR-OPS-008 — Go-live sign-off:** Confirm the person authorised to accept opening records, security permissions, known limitations and cutover.

What stays disabled until each answer is given

This section is informational. It exists so the cost of leaving an item blank is visible at the moment of deciding, rather than discovered later as a missing feature. The authoritative classification is in `14_UNRESOLVED_ITEM_BLOCKER_CLASSIFICATION.md`. *(C-09)*

UNANSWERED ITEM	WHAT REMAINS DISABLED
BR-REN-001 … BR-REN-006	Automated rent generation and billing runs. Charges can only be entered manually with a supplied amount.
BR-REN-008	Overdue status with penalties; any late-fee amount shown as authoritative.
BR-REN-012	Tax lines on any document; tax-invoice generation.
BR-PAY-002, BR-PAY-003	Marking a payment confirmed; any "confirmed collection" report figure.
BR-PAY-005	Automatic duplicate rejection on a UPI reference or bank UTR. Request-retry protection is unaffected and already works.
BR-ALLOC-002	Automatic allocation. Every allocation requires staff to choose the target charge explicitly.
BR-ALLOC-003, BR-ALLOC-004	Recording an unmatched receipt; holding advance credit.
BR-ALLOC-005	Reversing or reassigning an allocation.
BR-DEP-004 … BR-DEP-010	Deposit deductions, refunds and any final settlement calculation.
BR-DEP-006	Any netting of deposit against outstanding dues. This never happens automatically regardless.
BR-LIF-004, BR-LIF-011	Automatic bed release on checkout. Checkout can be recorded; the bed stays held.
BR-REP-001 … BR-REP-004	Authoritative totals on the corresponding report. Records are listed; the total is withheld.
BR-SEC-001 … BR-SEC-003	Staff account provisioning and go-live.
BR-SEC-004	Any document upload or download. Default is that none is built.

UNANSWERED ITEM	WHAT REMAINS DISABLED
BR-OPEN-001 … BR-OPEN-005	Real-data import and cutover.
BR-OPS-005	Any stated RPO, RTO or availability commitment.

A feature in this table is not broken and is not late. It is waiting for a decision only the owner can make.

Final sign-off

- ☐ All business rules for enabled P0 behaviours are answered and linked to test case IDs.
- ☐ Unresolved policies are explicitly disabled or remain manual with a safe, clearly labelled workflow.
- ☐ Opening-balance reconciliation is complete and exceptions are accepted in writing.
- ☐ The role and permission matrix is approved.
- ☐ Deposit and checkout policies are approved before any automated deduction, refund or final-settlement calculation is enabled.
- ☐ Operational checkout and financial settlement have been confirmed as separate workflows, and approving the first does not approve the second. *(C-07)*

Owner / authorised approver: _____ Signature / written approval reference: _____ Date: _____ Outstanding items and named owners: _____